

Než začneš psát vlastní funkce, zopakuj si, co je funkce v matematice, jak funguje cyklus s akumulátorem, jak se počítá faktoriál a kombinační číslo, jak ošetřit vstup podmínkou a jak funkci otestovat.

1. Funkce v matematice: předpis, argument, hodnota **SS**

Kde to najdeš: 1.+2. H11 Pojem funkce

Funkce přiřazuje každému vstupu (argumentu) právě jeden výstup (funkční hodnotu). Zápis $f(x) = 2x + 3$ je předpis: když za x dosadíš 4, dostaneš $f(4) = 11$. Stejný předpis použiješ pro libovolný vstup a nemusíš ho psát znovu. Podprogram v Pythonu funguje stejně: `def f(x): return 2 * x + 3` je předpis, `f(4)` je volání s argumentem 4 a výsledkem 11. Funkce může mít i více vstupů, třeba `kombinace(n, k)`.

$f(x) = 2x + 3$ předpis funkce

Příklad: Pro $f(x) = x^2 - 1$ urči $f(3)$, $f(-2)$ a $f(0)$ a zapiš funkci v Pythonu.

1. $f(3) = 9 - 1 = 8$.
2. $f(-2) = 4 - 1 = 3$.
3. $f(0) = 0 - 1 = -1$.
4. V Pythonu: `def f(x): return x ** 2 - 1`; volání `f(3)` vrátí 8.

Zkus sám:

1. Pro $g(x) = 3x - 5$ urči $g(2)$ a $g(-1)$.
2. Pro $h(a, b) = a \cdot b + 1$ urči $h(3, 4)$.

2. Cyklus a akumulátor **SS**

Kde to najdeš: 1.+2. H49 Cykly a seznamy

Cyklus `for` opakuje tělo pro každou hodnotu z `range` nebo ze seznamu. Akumulátor je proměnná, do které průběžně ukládáš výsledek: pro součet začíná nulou, pro součin jedničkou. Faktoriál je přesně takový součin: $p = 1$ a potom pro i v `range(1, n + 1)` provedeš $p = p \cdot i$. Pamatuj, že `range(1, n + 1)` končí číslem n .

Příklad: Co vypíše program `p = 1; for i in range(1, 6): p = p * i; print(p)`?

1. `range(1, 6)` dává čísla 1, 2, 3, 4, 5.
2. `p` se postupně mění: 1, 2, 6, 24, 120.
3. Program vypíše 120, tedy 5!.

Zkus sám:

1. Co vypíše program `s = 0; for i in range(1, 5): s = s + i; print(s)`?
2. Kolikrát proběhne cyklus pro `range(1, 11)`?

3. Faktoriál a kombinační číslo SS

Kde to najdeš: 1.+2. H50 Variace a permutace, 1.+2. H51 Kombinace, binomická věta

Faktoriál $n!$ je součin čísel od 1 do n a platí $0! = 1$. Kombinační číslo $\binom{n}{k}$ udává, kolika způsoby vybereš k prvků z n , když nezáleží na pořadí. Počítá se ze tří faktoriálů, rychlejší je ale součinný tvar: k klesajících činitelů v čitateli děleno $k!$. Platí $\binom{n}{0} = \binom{n}{n} = 1$ a $\binom{n}{k} = \binom{n}{n-k}$.

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \text{ kombinační číslo} \quad \binom{n}{k} = \frac{n(n-1) \dots (n-k+1)}{k!} \text{ součinný tvar}$$

Příklad: Vypočti $\binom{10}{3}$ oběma způsoby.

1. Z faktoriálů: $\frac{10!}{3! \cdot 7!} = \frac{3\,628\,800}{6 \cdot 5\,040} = 120$.
2. Součinný tvar: $\frac{10 \cdot 9 \cdot 8}{3 \cdot 2 \cdot 1} = \frac{720}{6} = 120$.
3. Druhý způsob potřebuje mnohem méně násobení.

Zkus sám:

1. Vypočti $5!$ a $\binom{6}{2}$.
2. Vypočti $\binom{20}{18}$.

4. Větvení a ošetření vstupu SS

Kde to najdeš: 1.+2. H44 Větvení: řešič rovnice

Funkce by měla zvládnout i nesmyslné vstupy. Kombinační číslo $\binom{n}{k}$ má smysl jen pro celá čísla $0 \leq k \leq n$; pro jiné hodnoty je rozumné vrátit 0, nebo ohlásit chybu. Kontrolu zapíšeš složenou podmínkou na začátku funkce, třeba `if k < 0 or k > n: return 0`. Výjimečné případy řešíš dřív než samotný výpočet – stejně jako $a = 0$ v řešiči kvadratické rovnice.

Příklad: Navrhni podmínku, která ve funkci kombinace(n , k) zachytí všechny nesmyslné vstupy pro celá čísla n a k .

1. Nesmyslné je záporné n nebo k a také k větší než n .
2. Podmínka: $n < 0$ or $k < 0$ or $k > n$.
3. Pro takový vstup funkce vrátí 0 – žádný výběr neexistuje.
4. Test: kombinace(3, 5) má vrátit 0, kombinace(5, 0) má vrátit 1.

Zkus sám:

1. Co vrátí výraz $0 \leq k \leq n$ pro $n = 4$ a $k = 6$?
2. Zapiš v Pythonu podmínku „ n je sudé a kladné“.

5. Test příkazem assert SS

Kde to najdeš: 1.+2. H47 Testování a ladění

Test porovná skutečný výsledek s předem známým. V Pythonu ho zapíšeš příkazem `assert`: `assert faktorial(5) == 120`. Když podmínka platí, program pokračuje bez hlášení; když neplatí, skončí chybou `AssertionError` a ty víš, kde hledat. Do testů zařaď běžné i hraniční vstupy – u faktoriálu 0 a 1, u kombinací $k = 0$ a $k = n$.

Příklad: Navrhni tři testy pro funkci faktorial(n).

1. Běžný případ: `assert faktorial(5) == 120`.
2. Hranice: `assert faktorial(0) == 1` (dohoda $0! = 1$).
3. Druhá hranice: `assert faktorial(1) == 1`.
4. Proběhnou-li všechny bez chyby, funkce pro tyto vstupy funguje.

Zkus sám:

1. Co se stane po příkazu `assert 2 + 2 == 5`?
2. Jaké dva hraniční testy přidáš funkci kombinace(n , k)?

Kde se to doučit

- **Definice funkcí – Nauč se Python (PyLadies)** – Jak napsat vlastní funkci s parametry a návratovou hodnotou `return`.

<https://naucese.python.cz/course/pyladies/beginners/def/>

- **Vrátit nebo vypsát? – Nauč se Python** – Proč je lepší výsledek z funkce vrátit než jen vypsát a jak to usnadní další použití.
<https://naucse.python.cz/course/pyladies/beginners/prefer-return/>
- **Umíme matiku – Kombinační čísla** – Výpočty kombinačních čísel s okamžitou kontrolou. Bez licence je počet odpovědí denně omezen.
<https://www.umimematiku.cz/cviceni-kombinacni-cisla>

Výsledky ke kontrole

- **1. Funkce v matematice: předpis, argument, hodnota:** 1) $g(2) = 1$, $g(-1) = -8$; 2) 13
- **2. Cyklus a akumulátor:** 1) 10; 2) desetkrát
- **3. Faktoriál a kombinační číslo:** 1) 120 a 15; 2) $\binom{20}{2} = 190$
- **4. Větvění a ošetření vstupu:** 1) False; 2) $n \% 2 == 0$ and $n > 0$
- **5. Test příkazem assert:** 1) program skončí chybou AssertionError; 2) např. $kombinace(6, 0) == 1$ a $kombinace(6, 6) == 1$